Package ‘scScale’

July 8, 2026

Title Single-Cell Scaling Law Utilities

Version 0.0.1

Author Jingyuan Hu [aut, cre],
Sijia Zhu [aut, cre]

Maintainer Jingyuan Hu <jingyuanhu@example.com>

Description Median Marchenko-Pastur noise calibration, BBP spike inversion, spectral overlap, and task-difficulty utilities for single-cell scaling analyses, aligned with ChesleaZ/scScale.

License MIT + file LICENSE

URL https://github.com/ChesleaZ/scScale

BugReports https://github.com/ChesleaZ/scScale/issues

Encoding UTF-8

Depends R (>= 2.10),
Matrix

LazyData true

LazyDataCompression xz

Suggests irlba,
SeuratObject

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

Contents

`gse123025_myeloid_hvg` *GSE123025 Myeloid HVG Count Matrix Example*

Description

A larger example subset derived from GSE123025 for RMT spike-model tutorials. The object keeps all cells from the source matrix and restricts genes to 2,000 highly variable genes selected by `select_hvgs()`.

Usage

```
data(gse123025_myeloid_hvg)
```

Format

A list with:

counts Integer matrix with 2,000 selected genes/features by 1,922 cells.

source Source file name in the canonical lab data directory.

accession GEO accession identifier, `"GSE123025"`.

description Short description of the HVG subset.

selection List describing the feature and cell selection.

original_dim Dimensions of the source matrix before HVG selection.

cell_totals Column sums of the HVG count matrix.

gene_totals Row sums of the HVG count matrix.

Source

Derived from `data/GSE123025/GSE123025_Single_myeloid_1922_cells_processed_data.csv.gz` in the canonical lab workspace. The source matrix has 23,429 genes by 1,922 cells and is about 8.0 MB compressed as CSV gzip, or 95 MB uncompressed.

Examples

```
data(gse123025_myeloid_hvg)
counts <- gse123025_myeloid_hvg$counts
dim(counts)
fit <- scscale_fit(counts, r = 10, mp_max_iter = 3, fit_umi = FALSE)
fit
```

gse164378_3p_citeseq_hvg *GSE164378 3 Prime CITE-seq RNA HVG and ADT Example*

Description

A compact paired RNA and ADT example derived from the 3 prime CITE-seq portion of GSE164378. The object keeps 4,000 matched PBMC cells, 2,000 RNA highly variable genes, and all 228 measured ADT features.

Usage

```
data(gse164378_3p_citeseq_hvg)
```

Format

A list with:

rna_counts Sparse integer matrix with 2,000 selected RNA genes by 4,000 cells.

adt_counts Sparse integer matrix with 228 ADT features by the same 4,000 cells.

meta Cell metadata, including RNA/ADT QC totals, lane, donor, time, batch, and cell type labels.

source Short source dataset description.

accession GEO accession identifier, `"GSE164378"`.

description Short description of the compact paired-modality subset.

selection List describing the cell and RNA feature selection.

original_dim Dimensions of the source 3 prime RNA and ADT matrices before subsetting.

cell_totals RNA and ADT column sums for the selected cells.

feature_totals RNA and ADT row sums for the selected features.

Source

Derived from the 3 prime RNA and ADT matrices in GSE164378, the Hao et al. multimodal PBMC reference dataset. The source 3 prime matrices have 33,538 RNA features by 161,764 cells and 228 ADT features by the same 161,764 cells.

Examples

```
data(gse164378_3p_citeseq_hvg)
rna <- gse164378_3p_citeseq_hvg$rna_counts
adt <- gse164378_3p_citeseq_hvg$adt_counts
dim(rna)
dim(adt)
```

scScale — *Single-Cell Scaling Law Utilities*

Description

Gaussian spike fitting and low-rank spectral mutual information utilities for single-cell scaling-law analyses.

Details

The recommended paired-modality workflow is:

1. Fit matched modalities once with `scscale_pair_fit`.
2. Fit UMI-depth scaling with `scscale_umi_mi`.
3. Fit cell-number scaling with `scscale_cell_number_mi`.
4. Fit the joint cell-number by UMI grid with `scscale_cell_number_by_umi_mi`.
5. Compare with direct empirical MI from `scscale_empirical_mi`.

All theory curves are routed through `scscale_low_rank_mi`, the single low-rank subspace-alignment MI formula.

See Also

`scscale_fit`, `scscale_pair_fit`, `scscale_low_rank_mi`, `scscale_umi_mi`, `scscale_cell_number_mi`, `scscale_cell_number_by_umi_mi`, `scscale_empirical_mi`

scscale_cell_number_by_umi_mi — *Fit Joint Cell-Number by UMI MI Scaling*

Description

Compute the theory MI grid over both cell number and UMI depth, reusing the refitted UMI spike strengths and subspace alignments.

Usage

```
scscale_cell_number_by_umi_mi(pair, umi, n_grid,
  sampling_rates = umi$sampling_rates)
```

Arguments

`pair`	A `scscale_pair_fit` object.
`umi`	A `scscale_umi_mi` object.
`n_grid`	Cell-number grid.
`sampling_rates`	Subset or ordering of UMI sampling rates to use.

Value

A data frame with `n`, `c_X`, `I_theory`, and `sampling_rate`.

`scscale_cell_number_mi` *Fit Cell-Number MI Scaling*

Description

Compute the theory MI curve as the number of cells varies.

Usage

```
scscale_cell_number_mi(pair, n_grid,
  q_X = pair$x_fit$spikes$q_X[seq_len(pair$r_X)], P = pair$P)
```

Arguments

`pair`	A `scscale_pair_fit` object.
`n_grid`	Cell-number grid.
`q_X`	Optional X-side spike-strength vector.
`P`	Optional X/Y low-rank subspace alignment matrix.

Value

A data frame with `n`, `c_X`, and `I_theory`.

`scscale_empirical_inu_grid` *Sample an Empirical I(n, U) Grid*

Description

Sample cells and sequencing depth, then compute fixed-rank empirical subspace MI for each `(n,U)` grid point. This is the direct empirical counterpart to spike-model scaling workflows.

Usage

```
scscale_empirical_inu_grid(x, target, n_grid, U_grid = NULL,
  sampling_rates = NULL, r = 10, n_replicates = 1, seed = 1,
  input = c("counts", "normalized"), target_input = input,
  target_depth = 1e4, count_transform = c("log1p_cpm", "pearson_residual",
  "log1p"), center = TRUE, scale = FALSE, eps = 1e-12,
  use_irlba = TRUE)
```

Arguments

`x`	Feature-by-cell source matrix.
`target`	Feature-by-cell target matrix or cell-level target vector.
`n_grid`	Cell numbers to sample.
`U_grid`	Target UMI depths per cell. Provide either `U_grid` or `sampling_rates`.
`sampling_rates`	Count downsampling fractions in `(0,1]`.
`r`	Fixed rank used for empirical subspace MI.
`n_replicates`	Number of replicate subsamples per grid point.
`seed`	Random seed base.
`input`, `target_input`	Input type for `x` and matrix targets.
`target_depth`, `count_transform`, `center`, `scale`	Preprocessing controls passed to empirical MI calculation.
`eps`	Numerical clipping value for MI.
`use_irlba`	Whether to use **irlba** for partial SVD when available.

Value

A data frame with sampled `n`, requested `U`, replicate index, `I_empirical`, effective MI rank, and observed UMI summaries.

`scscale_empirical_mi` *Compute Empirical Subspace Mutual Information*

Description

Compute empirical low-rank subspace MI directly from data, either between two matrices or between one matrix and a cell-level target.

Usage

```
scscale_empirical_mi(x, target, r = 10, input = c("counts", "normalized"),
  target_input = input, target_depth = 1e4, count_transform = c("log1p_cpm",
  "pearson_residual", "log1p"), center = TRUE, scale = FALSE,
  eps = 1e-12, use_irlba = TRUE, store_subspaces = FALSE)
```

Arguments

`x`	Feature-by-cell matrix for the source modality.
`target`	Second feature-by-cell matrix or cell-level target vector.
`r`	Number of singular-vector directions to keep.
`input`, `target_input`	Input type for `x` and matrix targets.
`target_depth`, `count_transform`, `center`, `scale`	Preprocessing controls.
`eps`	Numerical clipping value.
`use_irlba`	Whether to use **irlba** for partial SVD when available.
`store_subspaces`	Whether to keep fitted subspace matrices in the output.

Value

A list containing empirical `mi`, canonical-correlation-style `gamma`, and effective rank information.

---

scscale_empirical_scaling_fit
Fit Direct Empirical Scaling Laws

Description

Fit direct nonlinear scaling laws to fixed-rank empirical subspace MI values, without spike calling or Marchenko–Pastur bulk fitting. The high-level `scscale_empirical_scaling_fit()` function samples an empirical `I(n,U)` grid, preprocesses each sampled view, computes empirical MI, and fits the marginal and joint laws into one `scscale_empirical_inu_fit` object. The cell law is

$$I(n) = I_{\infty|U_0} - C_n \log(1 + A_n/n).$$

The UMI law is

$$I(U) = I_{\infty|n_0} - C_U \log(1 + A_U/(U + U_0)).$$

The joint law is additive in the two empirical information gaps:

$$I(n, U) = I_\infty - C_n \log(1 + A_n/n) - C_U \log(1 + A_U/(U + U_0)).$$

Usage

```
scscale_empirical_scaling_fit(x, target, n_grid, U_grid = NULL,
  sampling_rates = NULL, r = 10, n_replicates = 1, n_ref = NULL,
  U_ref = NULL, joint_mode = c("hybrid", "transfer", "free"), seed = 1,
  input = c("counts", "normalized"), target_input = input,
  target_depth = 1e4, count_transform = c("log1p_cpm", "pearson_residual",
  "log1p"), center = TRUE, scale = FALSE, eps = 1e-12,
  use_irlba = TRUE, cell_start = NULL, umi_start = NULL,
  joint_start = NULL, cell_control = list(maxit = 10000),
  umi_control = list(maxit = 10000),
  joint_control = list(maxit = 20000))

scscale_fit_empirical_cell_law(data, n_col = "n", mi_col = "I_empirical",
  start = NULL, control = list(maxit = 10000))

scscale_fit_empirical_umi_law(data, U_col = "U", mi_col = "I_empirical",
  start = NULL, control = list(maxit = 10000))

scscale_fit_empirical_joint_law(data, n_col = "n", U_col = "U",
  mi_col = "I_empirical", cell_fit = NULL, umi_fit = NULL,
  mode = c("hybrid", "transfer", "free"), start = NULL,
  control = list(maxit = 20000))

scscale_empirical_global_ceiling(cell_fit, umi_fit, n0, U0)

scscale_empirical_cell_curve(n, I_infinity, C_n, A_n)
```

```
scscale_empirical_umi_curve(U, I_infinity, C_U, A_U, U_offset)

scscale_empirical_joint_curve(n, U, I_infinity, C_n, A_n, C_U, A_U,
  U_offset)
```

Arguments

x	Feature-by-cell source matrix.
target	Feature-by-cell target matrix or cell-level target vector.
n_grid	Cell numbers to sample.
U_grid	Target UMI depths per cell. Provide either U_grid or sampling_rates.
sampling_rates	Count downsampling fractions in (0,1].
r	Fixed rank used for empirical subspace MI.
n_replicates	Number of replicate subsamples per grid point.
n_ref, U_ref	Reference cell number and UMI depth for marginal fits. When omitted, the largest sampled value is used. If a supplied reference is not on the grid, the nearest sampled grid value is used.
joint_mode	Joint fitting mode passed to scscale_fit_empirical_joint_law.
seed	Random seed base for grid sampling.
input, target_input	Input type for x and matrix targets.
target_depth, count_transform, center, scale	Preprocessing controls passed to empirical MI calculation.
eps	Numerical clipping value for MI.
use_irlba	Whether to use **irlba** for partial SVD when available.
cell_start, umi_start, joint_start	Optional named starting values for the marginal and joint nonlinear fits.
cell_control, umi_control, joint_control	Control lists passed to optim for the marginal and joint nonlinear fits.
data	Data frame containing empirical MI observations.
n_col, U_col, mi_col	Column names for cell number, UMI depth, and empirical MI.
start	Optional named list of starting values.
control	Control list passed to optim.
cell_fit, umi_fit	Marginal empirical scaling fits used to initialize or transfer joint-law parameters.
mode	Joint fitting mode. "transfer" fixes marginal gap parameters and fits only the global ceiling; "hybrid" fixes scale parameters and refits the ceiling and amplitudes; "free" refits all joint parameters.
n, U	Cell number and UMI depth values for curve evaluation.
n0, U0	Reference cell number and UMI depth for extrapolating the global ceiling from marginal fits.
I_infinity, C_n, A_n, C_U, A_U, U_offset	Scaling-law parameters.

Value

`scscale_empirical_scaling_fit()` returns an object of class `scscale_empirical_inu_fit`, containing the empirical grid, marginal fits, joint fit, reference values, ceiling diagnostics, residuals, and fit metrics. The lower-level fitting functions return objects of class `scscale_empirical_scaling_fit`, with fitted coefficients, residuals, fit metrics, and `predict()` support. `scscale_empirical_global_ceiling` returns the two marginal extrapolations to `I(infinity, infinity)` and their average.

`scscale_fit` *Fit a Gaussian Spike Model*

Description

Fit the input matrix with the Gaussian spike model, estimate MP bulk and spike parameters, and optionally attach target and UMI-scaling summaries.

Usage

```
scscale_fit(x, input = c("counts", "normalized", "representation",
  "eigenvalues"), target = NULL, target_fit = NULL, n = NULL, p = NULL,
  target_depth = 1e4, count_transform = c("log1p_cpm",
  "pearson_residual", "log1p"), center = TRUE, scale = FALSE,
  n_features = NULL, min_cells = 10, mp_max_iter = 300, mp_grid_n = 20000,
  mp_stop_metric = c("qq_rmse_log", "ks", "neg_log_likelihood", "none"),
  mp_stop_tol = 1e-4, mp_min_iter = 2, mp_gap_z_threshold = 6,
  mp_quiet_z_threshold = 5, mp_quiet_run = 20,
  mp_background_skip = 25, mp_background_window = 500,
  mp_central_quantiles = c(0.10, 0.70), r = NULL,
  force_spikes = FALSE,
  fit_umi = input[1] == "counts",
  sampling_rates = c(0.10, 0.20, 0.35, 0.50, 0.70, 0.85, 1.00),
  U_grid = NULL, umi_linear_intercept = NULL, umi_linear_slope = NULL,
  umi_reference_U = NULL, umi_clamp_rate = TRUE, umi_seed = 1,
  umi_replicates = 1, store_matrix = FALSE, use_irlba = TRUE)
```

Arguments

`x` Feature-by-cell counts, normalized matrix, representation matrix, or eigenvalue vector.

`input` Input type.

`target, target_fit`
: Optional target inputs used by legacy MI summaries.

`n, p` Cell and feature counts, required when `input = "eigenvalues"`.

`target_depth, count_transform, center, scale, n_features, min_cells`
: Count preprocessing controls.

`mp_max_iter, mp_grid_n, mp_stop_metric, mp_stop_tol, mp_min_iter, mp_gap_z_threshold, mp_quiet_z_threshold, mp_quiet_run, mp_background_skip, mp_background_window, mp_central_quantiles`
: MAD/MP bulk and spike-fitting controls.

`r` Optional number of spikes to keep.

`force_spikes` Whether to force the retained top components to be treated as spikes.

`fit_umi`, `sampling_rates`, `U_grid`, `umi_seed`, `umi_replicates`
: Legacy UMI scaling controls.

`umi_linear_intercept`, `umi_linear_slope`, `umi_reference_U`, `umi_clamp_rate`
: Optional controls for linearized UMI-rate calibration.

`store_matrix` Whether to store the normalized matrix in the returned fit.

`use_irlba` Whether to use **irlba** for partial SVD when available.

Value

An `scscale_fit` object containing the spectrum, MP bulk fit, spike table, recoverability vectors, and optional stored normalized matrix.

Examples

```
data(gse123025_myeloid_hvg)
x <- gse123025_myeloid_hvg$counts[1:100, 1:200]
fit <- scscale_fit(x, mp_grid_n = 600, mp_max_iter = 50)
head(fit$spikes)
```

`scscale_low_rank_mi` *Low-Rank Subspace Mutual Information*

Description

Compute the theory MI from X/Y recoverability and optional low-rank subspace alignment. This is the single theory formula used by the scaling helpers.

Usage

```
scscale_low_rank_mi(theta_X, theta_Y, P = NULL, eps = 1e-12)
```

Arguments

`theta_X`, `theta_Y`
: Recoverability vectors for the X and Y low-rank signal directions.

`P` Optional subspace alignment matrix between X and Y. If omitted, directions are matched by rank.

`eps` Numerical clipping value.

Value

A list with `mi`, `I_theory`, singular values `sigma`, squared canonical correlations `gamma`, and effective ranks.

Examples

```
scscale_low_rank_mi(c(0.5, 0.2), c(0.4, 0.1))$mi
```

scscale_mi — *Compare Two Fitted scScale Objects*

Description

Compare two fitted objects with `scscale_low_rank_mi` and optional legacy scaling summaries.

Usage

```
scscale_mi(fit, target_fit, n_grid = NULL, U_grid = NULL,
  sampling_rates = NULL, combine = TRUE, empirical = TRUE,
  store_empirical_subspaces = FALSE, use_irlba = TRUE, r = NULL,
  P = NULL, eps = 1e-12)
```

Arguments

- `fit, target_fit` — `scscale_fit` objects.
- `n_grid, U_grid, sampling_rates` — Optional cell-number and UMI grids.
- `combine` — Whether to return a combined legacy grid when both dimensions are supplied.
- `empirical` — Whether to compute empirical subspace MI from stored matrices.
- `store_empirical_subspaces` — Whether to keep empirical subspace matrices.
- `use_irlba` — Whether to use **irlba** for partial SVD when available.
- `r` — Optional number of directions to keep.
- `P` — Optional low-rank subspace alignment matrix.
- `eps` — Numerical clipping value.

Value

A list containing theory MI and optional empirical or scaling components.

scscale_pair_fit — *Fit Matched Modalities for MI Scaling*

Description

Fit the noisy input modality, optionally fit or directly construct the target side, compute right singular vectors, and align the low-rank subspaces.

Usage

```
scscale_pair_fit(x, y, count_transform = c("log1p_cpm",
  "pearson_residual", "log1p"), mp_max_iter = 300, mp_grid_n = 3000,
  y_mode = c("spike", "subspace", "auto"), y_rank = NULL,
  y_center = TRUE, y_scale = FALSE, y_input = c("auto", "matrix",
  "labels", "numeric"), auto_low_rank_max = 50, rank_tol = 1e-08,
  x_rank = NULL, x_force_spikes = FALSE, use_irlba = TRUE, ...)
```

Arguments

- `x` — Matched feature-by-cell count matrix for the noisy input modality.
- `y` — Matched target. Usually a feature-by-cell count matrix for a second noisy modality, but can also be a deterministic low-rank target matrix or label/numeric vector when `y_mode = "subspace"`.
- `count_transform` — Count normalization transform passed to `scscale_fit`.
- `mp_max_iter`, `mp_grid_n` — MAD/MP spike-fitting controls.
- `y_mode` — How to handle `y`. `"spike"` preserves the original paired-modality behavior and fits a spike model to `y`. `"subspace"` treats `y` as a deterministic target subspace with target recoverability equal to one. `"auto"` uses conservative low-rank/one-hot detection.
- `y_rank` — Optional target rank when `y_mode = "subspace"`. Defaults to the numerical rank after preprocessing.
- `y_center`, `y_scale` — Whether to center or scale deterministic target rows before taking right singular vectors.
- `y_input` — Target type for deterministic targets. `"auto"` infers matrix, label, or numeric-vector input.
- `auto_low_rank_max`, `rank_tol` — Controls for conservative automatic deterministic-target detection.
- `x_rank` — Optional number of input spikes to retain.
- `x_force_spikes` — Whether to force the retained input components to be treated as spikes.
- `use_irlba` — Whether to use **irlba** for partial SVD when available.
- `...` — Additional arguments passed to `scscale_fit`.

Value

A `scscale_pair_fit` object with `x_fit`, `y_fit`, `z_X`, `z_Y`, the overlap matrix `P`, baseline `mi`, and `I_infinity`. When `y_mode = "subspace"`, `y_fit` is a `scscale_subspace_fit` object and no MP spike model is fitted to the target.

Examples

```
data(gse164378_3p_citeseq_hvg)
rna <- gse164378_3p_citeseq_hvg$rna_counts[1:100, 1:250]
adt <- gse164378_3p_citeseq_hvg$adt_counts[1:35, 1:250]
pair <- scscale_pair_fit(rna, adt, mp_grid_n = 600, mp_max_iter = 50)
pair$I_infinity

labels <- factor(rep(c("A", "B"), each = 125))
label_pair <- scscale_pair_fit(
  rna,
  labels,
  y_mode = "subspace",
  y_input = "labels",
  mp_grid_n = 600,
  mp_max_iter = 50
)
label_pair$y_mode
```

`scscale_subspace_overlap_matrix` *Compute a Subspace Overlap Matrix*

Description

Compute the cross-product matrix between two low-rank cell subspaces after optional cell-name alignment. This matrix is used by `scscale_low_rank_mi`.

Usage

```
scscale_subspace_overlap_matrix(z_X, z_Y)
```

Arguments

`z_X, z_Y`	Cell-by-rank orthonormal subspace bases. If row names are present, bases are aligned by shared cell names.

Value

A numeric overlap matrix with rows corresponding to `z_X` directions and columns corresponding to `z_Y` directions.

`scscale_target_fit` *Construct a Deterministic Target Subspace Fit*

Description

Construct a deterministic low-rank target object for use in `scscale_pair_fit`. Unlike `scscale_fit`, this function does not fit an MP/BBP spike model. It computes the target right singular vectors directly and sets target recoverability to one.

Usage

```
scscale_target_fit(y, input = c("matrix", "labels", "numeric"),
  rank = NULL, center = TRUE, scale = FALSE, drop_empty = TRUE,
  rank_tol = 1e-08, use_irlba = TRUE)
```

Arguments

`y`	Target matrix, label vector, or numeric vector. Matrix inputs are feature-by-cell.
`input`	Target input type.
`rank`	Optional number of target singular vectors. Defaults to the numerical rank after preprocessing.
`center, scale`	Whether to center or scale target rows before computing the target subspace. Label targets are typically centered and not scaled.
`drop_empty`	Whether to drop unused label levels.
`rank_tol`	Numerical rank tolerance.
`use_irlba`	Whether to use **irlba** for partial SVD when available.

Value

A `scscale_subspace_fit` object with target matrix `matrix`, cell subspace `z`, rank `r`, and unit recoverability vectors `theta_X` and `theta_infinity`.

Examples

```
labels <- factor(rep(c("T", "B", "Mono"), length.out = 90))
target <- scscale_target_fit(labels, input = "labels")
target$r
```

---

scscale_umi_mi — *Fit UMI-Depth MI Scaling*

Description

Refit the X modality over UMI depth, recompute the X/Y subspace alignment, and return both the refitted theory MI curve and the refitted `I_infinity` bound.

Usage

```
scscale_umi_mi(pair, x_counts, sampling_rates = c(0.10, 0.20, 0.35,
  0.50, 0.70, 0.85, 1.00),
  count_transform = pair$count_transform %||% "log1p_cpm", seed = 1,
  mp_max_iter = 300, mp_grid_n = 2000, use_irlba = TRUE)
```

Arguments

`pair`	A `scscale_pair_fit` object.
`x_counts`	Original X-side count matrix to downsample and refit.
`sampling_rates`	Global UMI sampling fractions in `(0, 1]`.
`count_transform`	Count transform used for refits.
`seed`	Random seed for downsampling.
`mp_max_iter`, `mp_grid_n`	MAD/MP spike-fitting controls for refits.
`use_irlba`	Whether to use **irlba** for partial SVD when available.

Value

A `scscale_umi_mi` object with `curve`, `q_by_rate`, `fit_by_rate`, and `P_by_rate`.

Examples

```
data(gse164378_3p_citeseq_hvg)
rna <- gse164378_3p_citeseq_hvg$rna_counts[1:100, 1:250]
adt <- gse164378_3p_citeseq_hvg$adt_counts[1:35, 1:250]
pair <- scscale_pair_fit(rna, adt, mp_grid_n = 600, mp_max_iter = 50)
umi <- scscale_umi_mi(pair, rna, sampling_rates = c(0.5, 1),
  mp_grid_n = 500, mp_max_iter = 50)
umi$curve
```

Index